

Social Publishing Pipeline — Architecture & Delivery Plan

Oct 1, 2026 · @Shadman Arko

Executive summary

One Python codebase on one VPS publishes every approved post to eight platforms within two minutes of its scheduled time. Dagster is the only clock and Postgres is the only source of truth. You plan posts in a Google Sheet, drop media into a folder on your PC, and approve; the system reports back into the Sheet and Telegram.

Decision	Why it matters
The Sheet is where you write; Postgres is the record	Editing, sorting or breaking the Sheet never changes what has been approved
One post fans out into one variant per platform, each with its own state machine	A failure on X never blocks Instagram
Prepare ahead, publish at T	Uploads, transcodes and containers happen early, so slow work never makes a post late
Native scheduling (YouTube, Facebook) is a capability flag, not a separate code path	The scheduler has one flow; platforms differ only inside their adapters
Ports and adapters, with Dagster as a thin shell	Business rules are testable without Dagster, Google or any platform
Never post twice: an uncertain outcome is checked with the platform, never blindly retried	Platforms offer no idempotency keys, so blind retries create duplicates
Media travels PC → Google Drive → VPS, and the VPS copy is deleted after publishing	Your PC can be off at publish time and nothing opens a port at home
App secrets are encrypted in git with SOPS; OAuth tokens are encrypted in Postgres	No plaintext secret exists in the repo, the Sheet, the logs or the image
API approvals start in week 1	Meta review and the TikTok and YouTube audits take longer than the code

Scope

Version 1 schedules images, carousels, videos and short-form video to eight platforms from one Sheet, and nothing more. Comments, metrics and agent-written posts plug in later without changing this core.

Goals

- Publish feed images, carousels, videos and shorts to Instagram, Facebook, Threads, LinkedIn, X, YouTube, Reddit and TikTok.
- Give each platform its own caption, time and options, with defaults set once per post.
- Show status, live link and errors per platform in the Sheet and in Telegram.
- Make a ninth platform cost one adapter, one Sheet tab and one config entry.
- Stay tenant-aware from day one: every row carries `tenant_id`, and Dhaka Kacchi is tenant one.

Not in version 1

- Editing or deleting posts after they are live.
- Comments, DMs and community replies, which are a separate lifecycle.
- Collecting post metrics; stored post IDs make this a later add-on.
- Stories, X threads, polls, live video and paid promotion.
- Facebook Groups (no API) and any web UI beyond the Sheet, Telegram and the Dagster UI.

Constraints

- One engineer, one VPS, open-source software only.
- Data stays in the EU, and the existing GDPR Article 9 schema rule applies.
- Every platform's terms and approval process apply: official APIs only, no scraping.
- It builds on the existing stack: Dagster, Postgres, dbt, the Telegram bot and the LLM gateway.

Quality targets

The two targets that matter most are zero duplicate posts and 99% of posts live within 120 seconds of their slot. TikTok is measured from its accepted publish call, because its processing time is outside the system's control. Every target is computed from Postgres,

so the daily digest reports them with no extra tooling.

Attribute	Target	Measured as
Punctuality	99% of variants live within 120 s of their scheduled time	<code>published_at - publish_at</code> per variant
No duplicates	0 posts published twice	Reconciliation never finds a second post for a variant
No unapproved posts	0 posts without an approval event	Every <code>published</code> variant has an <code>approved</code> event before it
Fidelity	Published content always equals the approved snapshot	Snapshot hash re-checked inside the publish step
Detection	Telegram alert within 5 min of any failed, late or uncertain variant	Alert time minus event time
Outage detection	Alert within 10 min if the VPS or Dagster stops	External dead-man's switch
Recovery	Rebuild on a fresh VPS in under 2 h; RPO 24 h, with publish state recovered from the platforms	Quarterly restore drill
Maintainability	A new platform adapter in 3 days or less, contract tests included	Delivery log
Test depth	At least 90% line coverage in the domain and application layers	CI report
Cost	€0 in software licences; one VPS	Bill of materials

Engineering principles

Each principle maps to one concrete rule in this codebase and one check that fails the build when the rule is broken. The rule that matters most is the dependency rule: business logic never imports Dagster, Google, an HTTP client or the database driver.

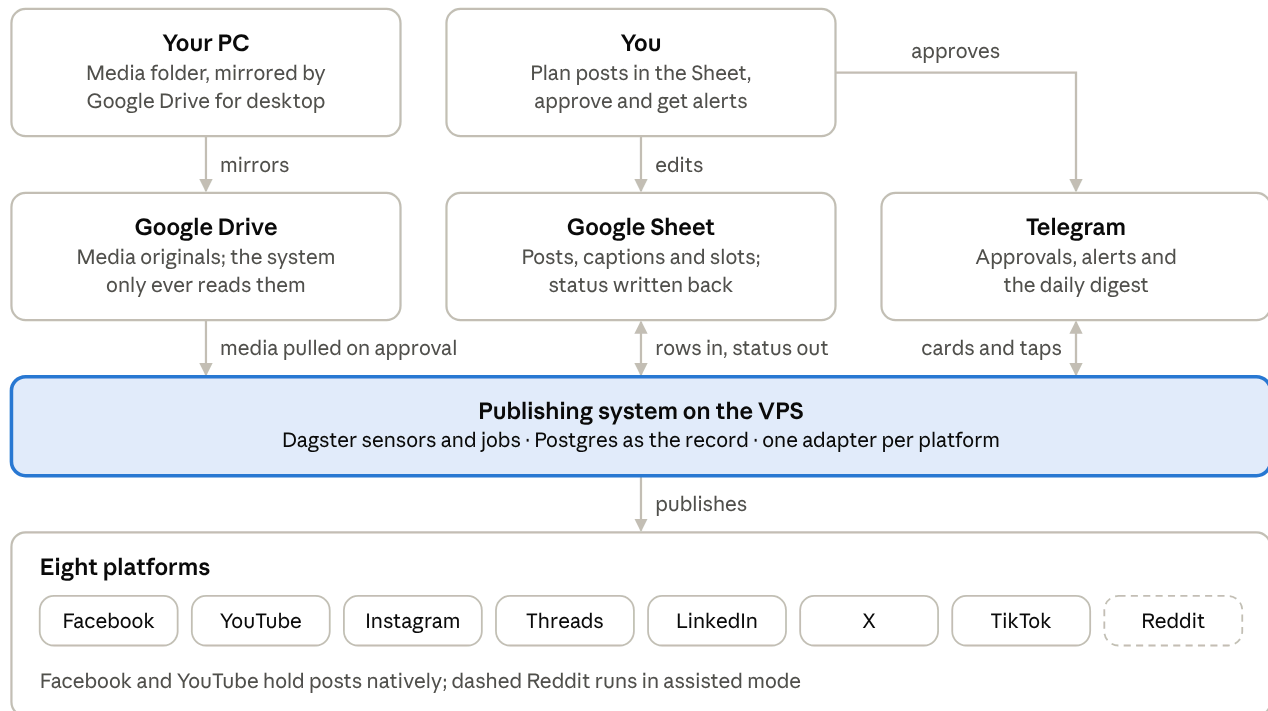
Principle	Concrete rule here	Enforced by
Single responsibility	One use case per module (sync, validate, approve, prepare, publish, reconcile, purge); one adapter per platform	Review checklist; modules over about 300 lines get flagged
Open/closed	Platforms register in one registry; the scheduler and use cases never name a platform	Architecture test: no platform names outside <code>adapters/platforms</code> and config
Liskov substitution	Every publisher looks the same to its callers; differences are declared as capabilities, never discovered through exceptions	One shared contract-test suite every adapter must pass
Interface segregation	Small ports: <code>Publisher</code> for all eight, <code>NativeScheduler</code> only for YouTube and Facebook	Type checker: an adapter implements only the protocols it declares
Dependency inversion	Use cases depend on <code>Protocol</code> ports; concrete adapters are wired in one composition root	import-linter layer contract in CI
KISS	One repo, one image, one database, one scheduler; Postgres doubles as the work queue	An ADR is required for any new runtime component
YAGNI	No live-post editing, plugin system or event sourcing until a real need appears	Scope section; ADRs
Clean code	Domain words in names (post, variant, rendition, slot); pure domain functions; explicit error types	ruff, mypy <code>--strict</code> , review
Twelve-factor	Config from the environment, stateless processes, logs to stdout, one Compose file for dev and prod	Container build and smoke test

When Dagster, Google Sheets or a platform API changes, only one adapter changes. That is the maintainability guarantee the rest of the design protects.

System context

The system deals with four outside parties: you, Google, Telegram and the eight platforms. Your PC feeds media into Google Drive but is never in the publish path, so it can be switched off at any time.

The VPS is the only part that talks to the platforms



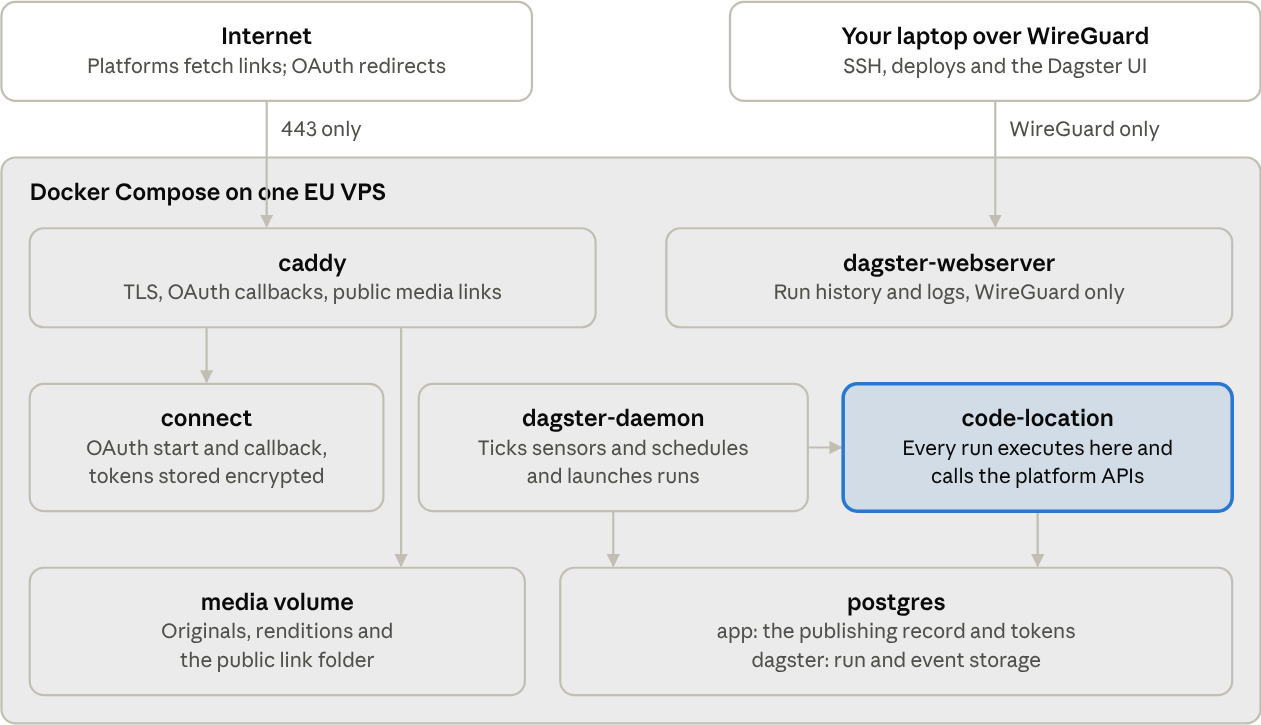
system context · you, Google, Telegram, the VPS and eight platforms

You edit the Sheet, drop media and tap Telegram; only the VPS ever calls a platform.

Runtime architecture

Everything runs as one Docker Compose project on one EU-hosted VPS, built from a single application image. Only Caddy is reachable from the internet; Postgres and the Dagster UI are reachable only over WireGuard.

Only Caddy faces the internet; everything else stays private



runtime · six services and a media volume, two ways in

Publishing runs inside the code-location container, and the only public entry point is Caddy on 443.

Service	Image	Role	Exposure
caddy	Caddy 2	TLS, reverse proxy, tokenised media URLs, OAuth callbacks	80/443, public
postgres	Postgres with pgvector	app database (publishing schema) and a separate dagster database for run storage	Internal network only
dagster-daemon	App image	Evaluates sensors and schedules, queues and launches runs	Internal
dagster-webserver	App image	Run history, logs and manual re-runs	WireGuard only

Service	Image	Role	Exposure
code-location	App image, dagster api grpc	Serves the Dagster definitions; every run executes here as a subprocess	Internal
connect	App image, FastAPI	OAuth start and callback endpoints for connecting accounts, plus /healthz	Through Caddy, admin-token protected
migrate	App image, one-shot	Applies schema migrations before each deploy	None

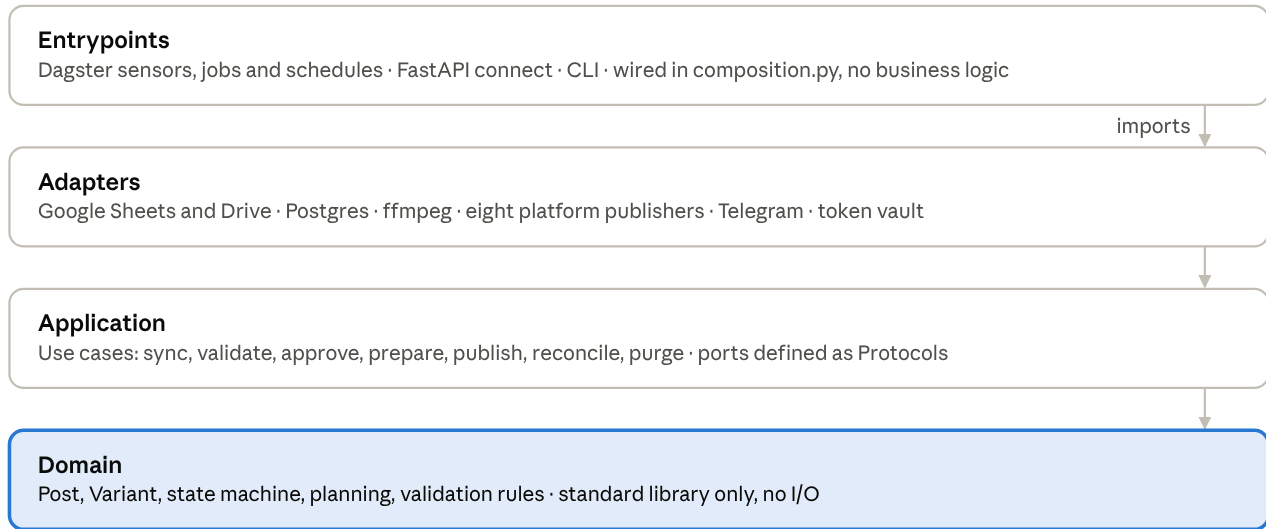
Three volumes hold state: `pgdata` for Postgres, `media` at `/srv/media` for originals, renditions and the public exposure folder, and `dagster_home` for instance config. Runs use Dagster's [default run launcher](#), which starts each run as a new process next to the job's code location. That keeps the Docker socket out of the containers; switch to a per-run container launcher only if a run ever needs isolation.

The firewall opens 80 and 443 for Caddy and 51820/udp for WireGuard. SSH listens only on the WireGuard interface. A 4 vCPU, 8 GB RAM, 160 GB disk VPS is a comfortable starting size for tenant one, including ffmpeg transcodes (an estimate; watch the CPU graph during the first renders).

Code architecture

The code follows ports and adapters: a pure domain core, use cases that depend only on ports, adapters at the edge, and Dagster as one entry point among several. Dependencies point inward only, and CI rejects any import that points outward.

Dependencies point inward; the domain imports nothing



Each arrow is an allowed import; CI rejects any import that points upward

code layers · four layers, imports point inward

Swapping Dagster, Google or a platform API touches only the top two layers.

Repository layout

```

publishing/
├─ pyproject.toml           # uv-managed, Python 3.12
├─ config/
│   └─ platforms.yaml      # lead times, lateness, limits per platform
(no secrets)
│   └─ renditions.yaml     # media profiles per platform and format
├─ migrations/            # SQL migrations, expand/contract only
├─ src/dk_publishing/
│   └─ domain/
│       └─ model.py        # pure Python: no I/O, no frameworks
│       └─ status.py       # Post, Variant, MediaAsset, Rendition, Slot
│       └─ planning.py     # VariantStatus and allowed transitions
│                           # native vs prepare/publish, prepare_at,
deadlines
│   └─ validation/        # one rules module per platform
│   └─ errors.py          # Retryable, RateLimited, AuthFailed,
Rejected, UnknownOutcome
│   └─ application/
│       └─ ports.py        # Protocols: repositories, Publisher,
NativeScheduler, MediaStore ...
│   └─ use_cases/         # sync_sheet, validate, approve, prepare,

```

```

publish, reconcile, cancel, purge, refresh_tokens
|   └─ adapters/
|   |   └─ sheets/           # Google Sheets reader and status writer
|   |   └─ media/           # Google Drive source, local media store,
ffmpeg transcoder
|   |   └─ persistence/     # Postgres repositories, unit of work
|   |   └─ platforms/       # one module per platform + registry
|   |   └─ secrets/         # token vault, the only importer of
cryptography
|   |   └─ notify/          # Telegram
|   └─ entrypoints/
|   |   └─ dagster_defs/    # sensors, schedules, jobs, resources (thin)
|   |   └─ connect_api/     # FastAPI OAuth flows
|   |   └─ cli.py           # admin commands
|   └─ composition.py       # the only place adapters are constructed
└─ tests/  unit/  contract/  integration/  sandbox/

```

The core ports

Every platform implements `Publisher`. Only platforms that can hold a post until a future time also implement `NativeScheduler`. Differences between platforms are declared in `Capabilities`, so the scheduler reads data instead of branching on platform names.

```

class Publisher(Protocol):
    platform: Platform
    capabilities: Capabilities

    def validate(self, v: VariantSnapshot) -> list[Violation]: ...
    def prepare(self, v: VariantSnapshot, media: Sequence[Rendition]) ->
PreparedHandle: ...
    def publish(self, handle: PreparedHandle) -> LivePost: ...
    def find_live(self, v: VariantSnapshot, handle: Handle | None) ->
LivePost | None: ...

class NativeScheduler(Protocol):
    def schedule(self, v: VariantSnapshot, media: Sequence[Rendition], at:
datetime) -> NativeHandle: ...
    def cancel(self, handle: NativeHandle) -> None: ...

@dataclass(frozen=True)
class Capabilities:
    native_window: tuple[timedelta, timedelta] | None # min and max lead the

```

```
platform accepts
    prepare_lead: timedelta                # start preparing
this long before the slot
    prepared_ttl: timedelta | None        # how long a prepared
handle stays valid
    pulls_media_by_url: bool              # platform fetches
media from a public URL
    max_lateness: timedelta               # past this, never
publish; expire instead
```

A platform with nothing to prepare returns a handle that simply carries its renditions. That keeps every adapter substitutable without empty methods that raise.

Rules CI enforces

Rule	Check
<code>domain</code> imports only the standard library	import-linter layers contract
<code>application</code> imports only <code>domain</code>	import-linter layers contract
Nothing imports <code>entrypoints</code> ; only <code>composition.py</code> constructs adapters	import-linter forbidden contract
Only <code>adapters/secrets</code> imports <code>cryptography</code>	import-linter forbidden contract
Platform names appear only in <code>adapters/platforms</code> and <code>config/</code>	Architecture test in pytest
Only the LLM gateway imports model SDKs (existing rule)	Existing CI test

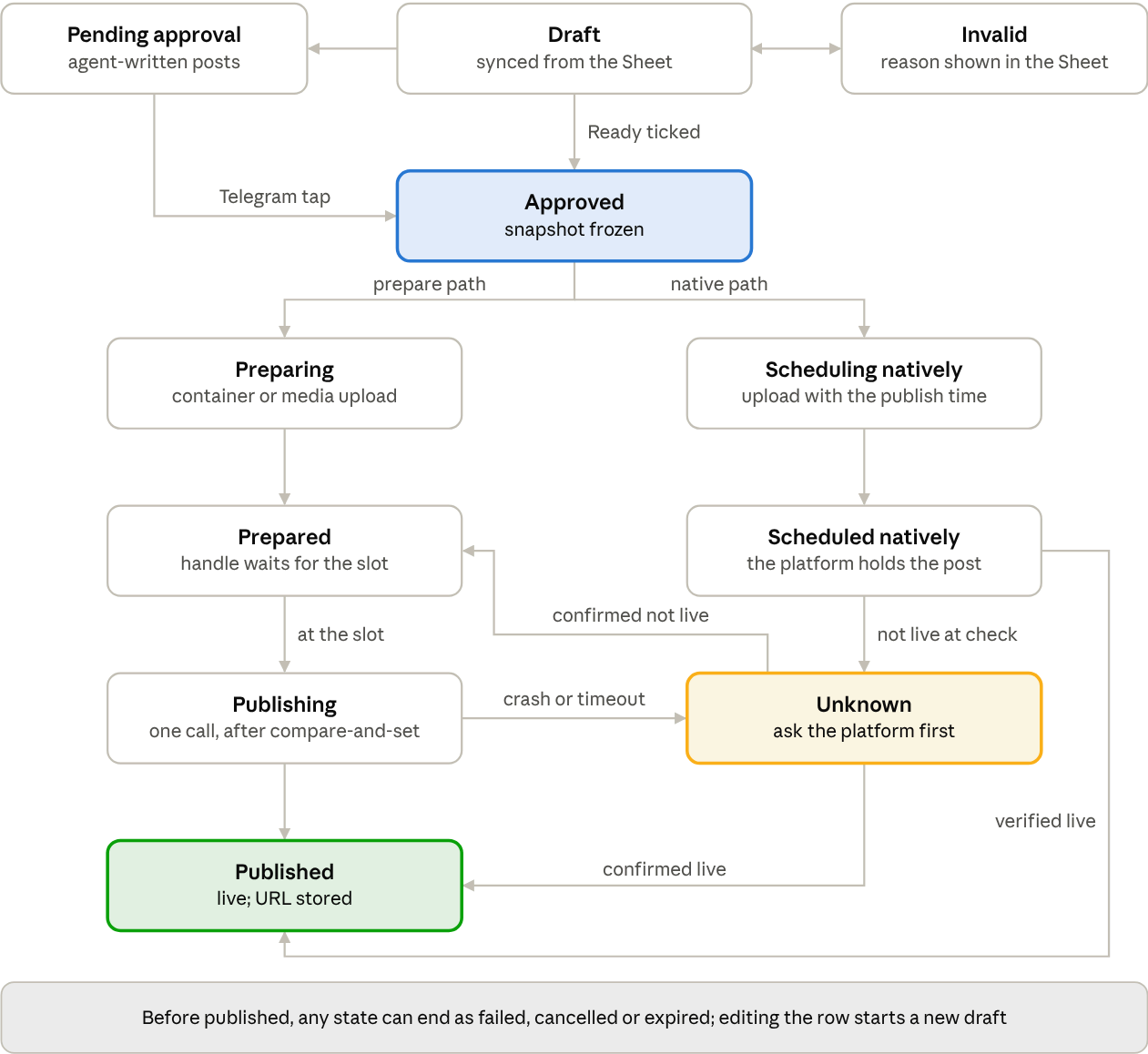
A Dagster op body stays three lines long: build the use case from the composition root, call it with a variant ID, and log the result. Business logic never lives in a sensor, a job or an op.

Domain model and variant lifecycle

A post is one idea with its media; a variant is that post on one platform account, and the variant is what gets scheduled, published and tracked. Every variant moves through one state machine, and only the domain's `transition()` function may change its state.

Entity	Meaning	Identity
Post	One idea: media, default caption, default slot	(tenant_id, post_key) from the Sheet
Variant	The post on one account: caption, slot, options, state	(post_id, platform, account_id)
Snapshot	The frozen content approved for a variant	SHA-256 of its canonical JSON
MediaAsset	One original file pulled from Drive	SHA-256 of its bytes
Rendition	A media file shaped for one platform profile	(asset_id, profile)
Attempt	One call to a platform in one phase	Unique idempotency key
SocialAccount	A page, channel, profile or handle you publish as	(tenant_id, platform, external_id)

Only an approved snapshot can reach published



variant lifecycle · two paths to published, one way back from unknown

Every route into Published starts at Approved, and an uncertain outcome is settled by asking the platform, never by retrying.

State	Meaning	Can move to
<code>draft</code>	Synced from the Sheet, not submitted	<code>invalid</code> , <code>pending_approval</code> , <code>approved</code> , <code>cancelled</code>

State	Meaning	Can move to
<code>invalid</code>	Failed validation; the reason is shown in the Sheet	<code>draft</code> on the next edit
<code>pending_approval</code>	Valid, waiting for a Telegram approval (agent-written posts)	<code>approved</code> , <code>draft</code> , <code>cancelled</code>
<code>approved</code>	Snapshot frozen; waiting for its prepare time	<code>preparing</code> , <code>scheduling_native</code> , <code>cancelled</code> , <code>expired</code>
<code>preparing</code>	Upload or container creation in progress	<code>prepared</code> , <code>approved (retry)</code> , <code>failed</code>
<code>prepared</code>	Handle ready; waiting for the slot	<code>publishing</code> , <code>approved (handle expired)</code> , <code>cancelled</code> , <code>expired</code>
<code>scheduling_native</code>	Uploading to a platform that holds the post until the slot	<code>scheduled_native</code> , <code>approved (retry)</code> , <code>failed</code>
<code>scheduled_native</code>	The platform will publish at the slot	<code>published</code> , <code>unknown</code> , <code>cancelled</code>
<code>publishing</code>	Publish call in flight	<code>published</code> , <code>unknown</code> , <code>prepared (retry)</code> , <code>failed</code>
<code>unknown</code>	Outcome uncertain after a crash or timeout	<code>published</code> , <code>prepared</code> , <code>failed</code>
<code>published</code>	Live; URL and platform ID stored	Terminal
<code>failed</code> , <code>cancelled</code> , <code>expired</code>	Terminal; editing the row creates a fresh draft	<code>draft</code> via an edit

Edits after approval

An edit before approval only updates the draft. An edit after approval but before the post is live invalidates the snapshot, cancels any native schedule, and sends the variant back to `draft`. Your own rows keep **Ready** ticked, so they re-validate and re-approve on the

next sync; agent-written rows wait for a new approval. Edits to a live post are ignored, and the Sheet says so.

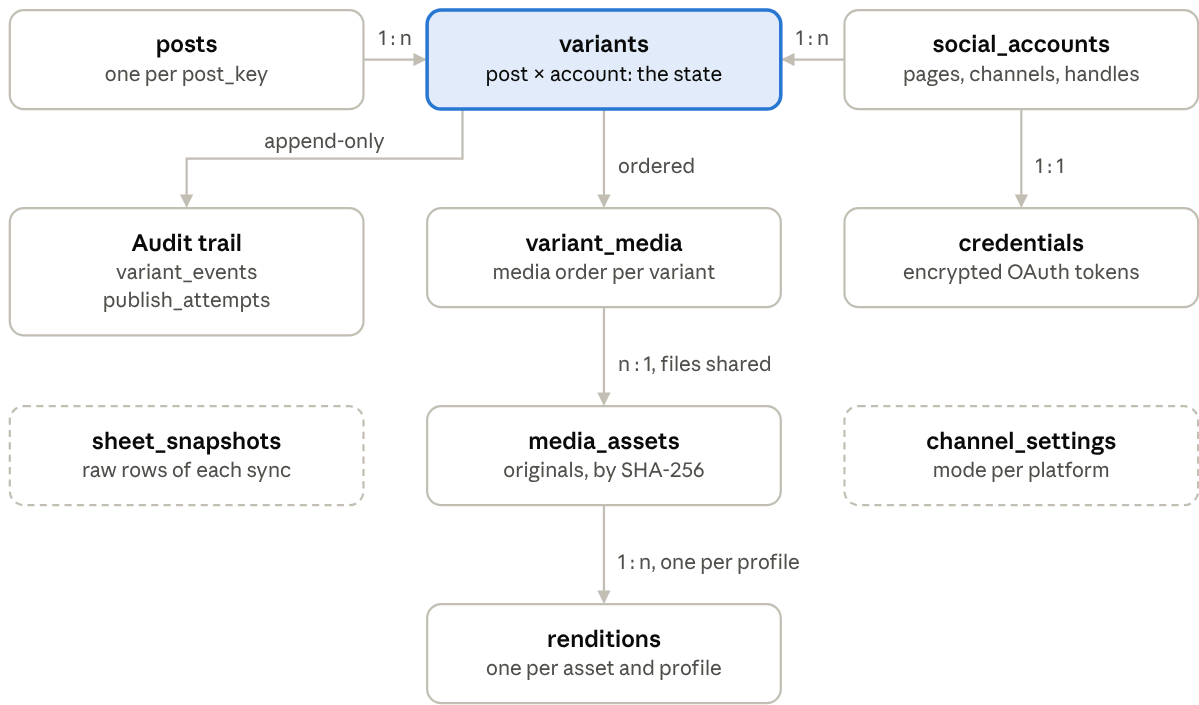
Invariants

1. No path reaches `published` without passing `approved`.
2. The publish step recomputes the snapshot hash and refuses to send anything else.
3. At most one attempt per variant is in flight: entering `publishing` is a compare-and-set on `(status, version)`.
4. `unknown` is never retried directly; reconciliation decides first.
5. All times are stored as UTC `timestampz`; Berlin local time exists only in the Sheet and in messages.
6. Every transition appends an event with actor and reason, and events are never updated or deleted.

Data model

Eleven tables in a `publishing` schema hold everything, and the scheduler only ever reads one indexed column: `variants.next_action_at`. The domain recomputes `next_action` and `next_action_at` on every transition, so the scheduling query needs no logic of its own.

Every variant hangs off one post and one account



Dashed tables stand alone · every table carries tenant_id

publishing schema · eleven tables around variants

Arrows follow the path from a post to its files; the two dashed tables have no foreign keys.

Table	Holds	Keys and notable columns
social_accounts	Each page, channel, profile or handle	Unique (tenant_id, platform, external_id); status is active, needs_reauth or paused
credentials	Encrypted OAuth tokens, one row per account	PK account_id; ciphertext, key_version, access_expires_at, refresh_expires_at, scopes; readable only by the vault's database role
sheet_snapshots	Raw rows of every sync that changed something, append-only	(sync_id, tab, row_index); values jsonb, row_hash
posts	One row per post	Unique (tenant_id, post_key); source is human or agent

Table	Holds	Keys and notable columns
<code>variants</code>	One row per post and account	Unique (<code>post_id</code> , <code>platform</code> , <code>account_id</code>); <code>status</code> , <code>version</code> , <code>publish_at</code> , <code>snapshot</code> , <code>snapshot_hash</code> , <code>next_action</code> , <code>next_action_at</code> , <code>native_handle</code> , <code>external_id</code> , <code>external_url</code> , <code>attempts</code>
<code>variant_media</code>	Ordered media per variant, for carousels	PK (<code>variant_id</code> , <code>position</code>)
<code>variant_events</code>	Every state change, append-only	PK (<code>variant_id</code> , <code>seq</code>); <code>from_status</code> , <code>to_status</code> , <code>actor</code> , <code>reason</code> , <code>at</code>
<code>publish_attempts</code>	Every platform call, append-only	Unique <code>idempotency_key</code> ; <code>phase</code> , <code>outcome</code> , <code>error_code</code> , <code>http_status</code> , redacted response excerpt
<code>media_assets</code>	Originals pulled from Drive	Unique (<code>tenant_id</code> , <code>sha256</code>); <code>drive_file_id</code> , <code>drive_md5</code> , <code>mime</code> , <code>bytes</code> , <code>width</code> , <code>height</code> , <code>duration_ms</code> , <code>purged_at</code>
<code>renditions</code>	Platform-shaped files	Unique (<code>asset_id</code> , <code>profile</code>); <code>path</code> , <code>sha256</code> , <code>public_token</code> , <code>exposed_until</code> , <code>purged_at</code>
<code>channel_settings</code>	Per tenant and platform switches	PK (<code>tenant_id</code> , <code>platform</code>); <code>mode</code> is off, <code>dry_run</code> , <code>assisted</code> or <code>live</code>

Every table carries `tenant_id`, and every unique key includes it, so tenant two needs no schema change. Postgres row-level security on `tenant_id` gets switched on before a second tenant arrives. Migrations follow expand-then-contract, so the previous release always runs against the new schema and a rollback is safe.

A partial index on `variants` (`next_action_at`) covering only non-terminal states keeps the scheduler's query to a few milliseconds however much history accumulates.

Google Sheet design

A `Posts` tab holds each idea once, and one tab per platform holds that platform's caption, time and options. You edit the white columns; grey columns belong to the system and are protected. Rows are matched by `post_key`, never by row number, so sorting and filtering are always safe.

Tabs

Tab	Purpose	Written by
<code>README</code>	How to use the Sheet, with examples	You
<code>Posts</code>	One row per post: key, media, default caption, default slot, Ready	You
<code>Instagram ... TikTok</code> (8 tabs)	One row per post and account on that platform	You (white), system (grey)
<code>Calendar</code>	Read-only view of everything scheduled, including agent-written posts	System
<code>_lists</code>	Dropdown values: accounts, formats, subreddits	System

Posts tab

Column	Example	Rule
<code>post_key</code>	<code>DK-2026-0412</code>	Required, unique, never reused or renamed
<code>title</code>	Eid platter reel	Internal name, never published
<code>media</code>	<code>eid_platter.mp4</code>	File names in the Drive media folder, comma-separated in display order
<code>default_caption</code>	—	Used by any platform tab whose caption is blank
<code>default_slot</code>	14.11.2026 18:00	Berlin local time, entered as a real date-time cell

Column	Example	Rule
<code>ready</code>	Checkbox	Ticking it submits the post, and for your own rows also approves it
<code>status</code> (grey)	5 of 8 live	Roll-up across platforms
<code>last_error</code> (grey)	Media file not found in Drive	Plain-language reason

Columns on every platform tab

Column	Rule
<code>post_key</code>	Dropdown of existing keys
<code>enabled</code>	Checkbox; unticking cancels anything not yet live
<code>account</code>	Dropdown of connected accounts on this platform
<code>caption</code>	Blank means the default caption
<code>slot</code>	Blank means the default slot
<code>status</code> , <code>live_url</code> , <code>last_error</code> , <code>synced_at</code> (grey)	Written back by the system

Platform-specific columns

Tab	Extra columns
Instagram	<code>format</code> (feed, carousel, reel), <code>share_to_feed</code> , <code>cover_at_s</code>
Facebook	<code>format</code> (post, photo, video, reel), <code>link</code>
Threads	<code>format</code> , <code>reply_control</code>
LinkedIn	<code>visibility</code>
X	<code>reply_settings</code>

Tab	Extra columns
YouTube	<code>title</code> , <code>description</code> , <code>tags</code> , <code>category</code> , <code>visibility_after_publish</code> , <code>made_for_kids</code> (required)
Reddit	<code>subreddit</code> , <code>title</code> , <code>flair</code> , <code>post_type</code>
TikTok	<code>privacy_level</code> (chosen per post, never defaulted), <code>allow_comments</code> , <code>allow_duet</code> , <code>allow_stitch</code> , <code>commercial_disclosure</code>

Sheet settings

- Set the spreadsheet time zone to Europe/Berlin. Slots must be date-time cells; text dates are rejected.
- Put data validation on every dropdown, checkbox and date column, and colour `status` with conditional formatting.
- Protect grey columns so only the service account edits them; you get a warning instead of a block.
- The sync finds columns by header name, so reordering columns is safe but renaming a header is not.

How the sync reads it

Every 2 minutes a sensor checks the spreadsheet's Drive `modifiedTime`, and only a change starts a sync run. The run reads every tab in one `values.batchGet`, stores the raw rows in `sheet_snapshots`, and parses each row into a typed model. It diffs against Postgres by `(post_key, platform, account)` and applies creates, edits and cancellations through the use cases. Status goes back in one `values.batchUpdate`, and grey columns are left out of the row hash, so the write-back never triggers another change.

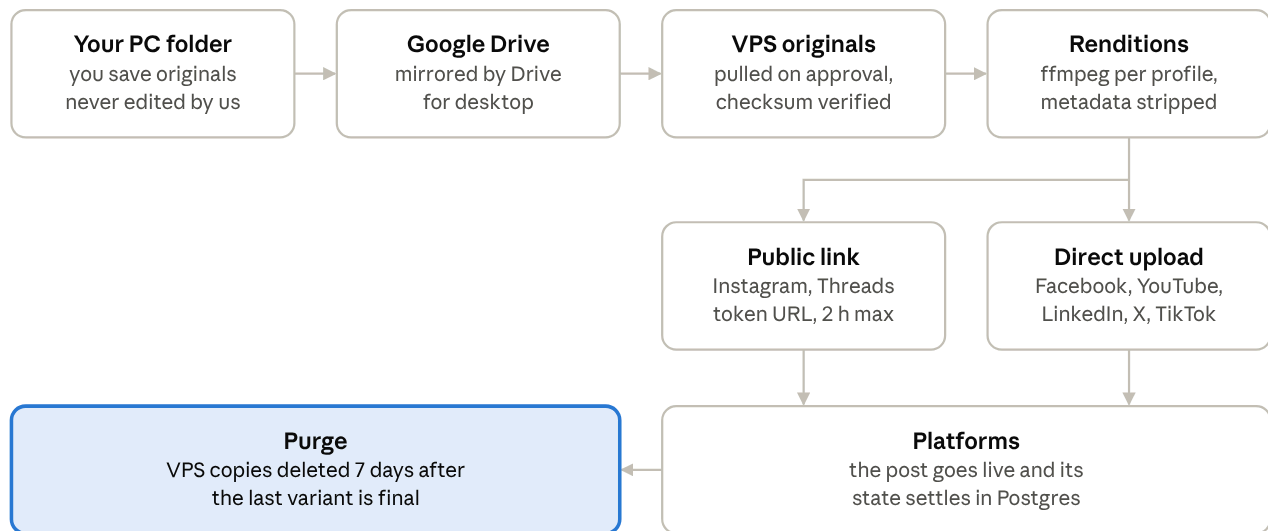
Validation errors are written in plain words, for example "Caption is 2,315 characters; this platform allows fewer." A slot in the past is invalid. A Berlin time that does not exist or happens twice because of daylight saving (02:30 on 29 March or 25 October 2026) is rejected rather than guessed.

One safety valve protects against slips: if a single sync would cancel more than five scheduled variants, it applies nothing and asks you in Telegram first. Deleting a block of rows by accident therefore never wipes a week of posts.

Media lifecycle

Media moves one way: PC folder → Google Drive → VPS → platforms, and the VPS copy is deleted 7 days after the last variant using it reaches a final state. The system never edits or deletes your originals on the PC or in Drive.

Your PC can be off at publish time; VPS copies are deleted after 7 days



media lifecycle · PC to Drive to VPS to platforms, then purge

Your originals stay on the left; everything on the VPS is temporary.

1. Save files into the media folder on your PC. Google Drive for desktop mirrors it to Drive, and a file appears in Drive only once its upload has finished, so half-copied files are never picked up.
2. Put the file name in the Sheet's `media` column. At sync, the validator resolves each name to a Drive file ID inside the configured folder; a missing name, or two files sharing a name, becomes a Sheet error.
3. When a variant is approved, `fetch_media` downloads the original to `/srv/media/originals/<sha256>`, checks it against Drive's MD5 checksum, and probes it with `ffprobe` or `Pillow`. A file used by several posts is stored once.

4. `render_media` builds each rendition defined in `config/renditions.yaml` with ffmpeg: H.264 and AAC in MP4 with the index at the front for video, sRGB JPEG for images, cropped or padded to the target ratio. Location and camera metadata are stripped from every file.
5. Instagram and Threads fetch media from a public URL at the moment of the call ([Instagram](#), [Threads](#)). For them, the prepare step links the rendition into `/srv/media/public/<random-token>/`, which Caddy serves. The link is removed once the platform reports the container ready, or after 2 hours at most.
6. Facebook, YouTube, LinkedIn, X and TikTok receive the file as a direct upload, so nothing is exposed publicly for them.
7. A daily `purge_media` job deletes originals and renditions from the VPS once every variant using them is `published`, `cancelled`, `failed` or `expired` and 7 days have passed. The metadata rows (hash, size, dimensions) stay for audit and deduplication.
8. Disk use above 70% raises an alert; above 85%, new fetches pause until space is freed.

If a Drive file's content changes after approval, the checksum changes and every variant using it returns to `draft` for re-approval. Remotion renders can be written straight into the Drive folder, so generated videos travel the same path as filmed ones.

Rendition profiles

Profiles are data, not code, so a new format needs no code change. Hard platform limits (duration, file size, carousel count) live in `platforms.yaml` and are checked during validation, before anything is rendered.

```
reel_9x16:
  kind: video
  size: 1080x1920
  fit: crop                # or pad
  video: {codec: h264, profile: high, faststart: true}
  audio: {codec: aac, sample_rate: 48000, channels: 2}
feed_4x5:
  kind: image
  size: 1080x1350
  format: jpeg
  colorspace: srgb
```

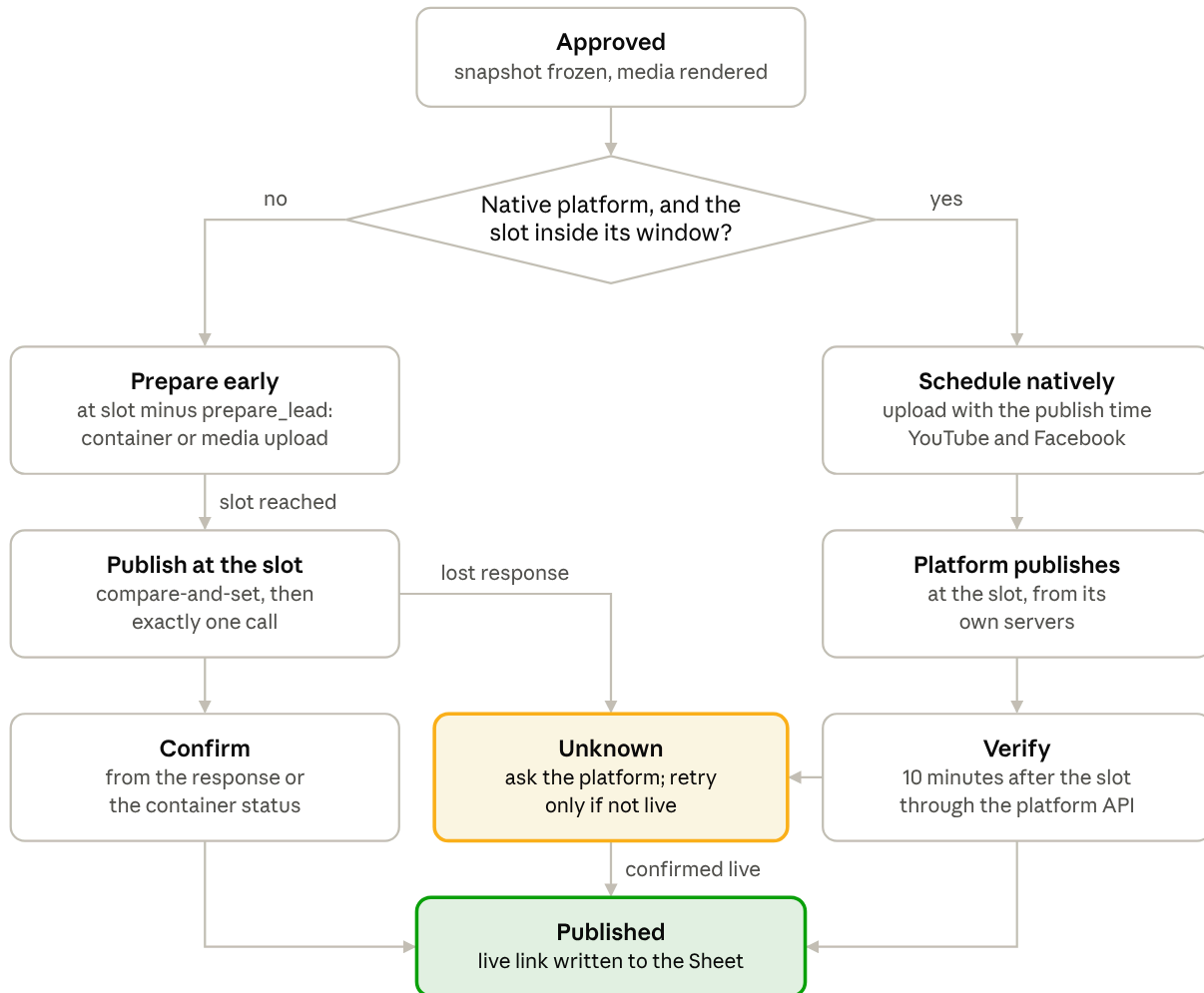
Why Drive instead of a direct PC-to-VPS sync

Drive keeps the PC out of the publish path: it can be asleep at 18:00 and the post still goes out. Nothing listens for inbound connections at home, the same service account already reads the Sheet, and you can add media from your phone. The `MediaSource` port keeps the alternative cheap: an rclone push over WireGuard is one new adapter if Drive is ever unwanted.

Scheduling and publishing engine

One sensor scans one indexed column every 30 seconds and launches one small job per due action; all timing and retry rules live in the domain, not in Dagster. Typical lateness stays under a minute: up to 30 s of sensor interval, a few seconds of run start-up, then the API call.

Every approved variant takes one of two paths, and both end in a check



publishing engine · native path and prepare path, one decision

Facebook and YouTube take the right-hand path, every other platform takes the left, and a lost response always passes through Unknown.

Dagster definitions

Definition	Kind	Cadence	Does
<code>sheet_changed</code>	Sensor	120 s	Compares the Sheet's Drive <code>modifiedTime</code> ; on change launches <code>sync_sheet</code> with the timestamp as run key

Definition	Kind	Cadence	Does
<code>due_actions</code>	Sensor	30 s	Reads up to 50 variants with <code>next_action_at <= now()</code> ; one run per action, run key <code><action>: <variant_id>:<version></code>
<code>sync_sheet</code>	Job	On change	Snapshot, diff, validate, approve, cancel, write status back
<code>fetch_and_render</code>	Job	On approval	Pulls originals from Drive and builds renditions
<code>prepare_variant</code>	Job	At prepare time	Creates containers or uploads media, stores the handle
<code>schedule_native</code>	Job	After approval	Uploads to YouTube or Facebook with the future publish time
<code>publish_variant</code>	Job	At the slot	Compare-and-set to <code>publishing</code> , then the single publish call
<code>reconcile_variant</code>	Job	After the slot, or on <code>unknown</code>	Asks the platform what happened and settles the state
<code>housekeeping</code>	Schedule	Every 5 min	Expires late variants, flags stale <code>publishing</code> rows, re-prepares expiring handles, pings the heartbeat
<code>refresh_tokens</code>	Schedule	Hourly	Refreshes tokens close to expiry
<code>purge_media</code>	Schedule	03:30 daily	Deletes VPS media past retention
<code>daily_digest</code>	Schedule	08:00 daily	Today's schedule, yesterday's results, anything waiting on you

Every schedule sets `execution_timezone="Europe/Berlin"`, so daylight saving never shifts the digest or the purge.

How the next action is chosen

After every transition, `planning.py` computes `next_action` and `next_action_at` from the variant and its platform's capabilities.

1. Native path: if the adapter implements `NativeScheduler` and the slot is inside its window, schedule natively once media is rendered. Facebook accepts 10 minutes to 30 days ahead ([Pages API](#)); a slot further out waits until it enters the window, and a slot closer than 10 minutes falls back to the prepare path.
2. Prepare path: otherwise prepare at `slot - prepare_lead` and publish at the slot. `prepare_lead` must stay below the handle's lifetime; Instagram containers expire after 24 hours ([Instagram](#)).
3. Verify: a natively scheduled variant gets a `reconcile` action 10 minutes after its slot.
4. Deadline: a variant that cannot publish within `max_lateness` (default 2 hours) expires and alerts you. An iftar post at 03:00 is worse than no post.

Never posting twice

Publish calls carry no idempotency key, so a blind retry after a lost response creates a duplicate. Five layers prevent that:

1. Run keys: the sensor never launches the same action twice.
2. Compare-and-set: `UPDATE variants SET status = 'publishing', version = version + 1 WHERE id = $1 AND status = 'prepared' AND version = $2`. Zero rows updated means another run owns it.
3. Intent log: the attempt row and its unique idempotency key are committed before the external call.
4. Stale detection: housekeeping turns a `publishing` row older than 10 minutes into `unknown`.
5. Reconciliation: `unknown` asks the platform first. Instagram reports a container as `PUBLISHED`, Threads has the same container status check, TikTok has a publish-status endpoint, and the rest are matched against the account's recent posts. Only a confirmed "not live" sends the variant back to `prepared`; if the platform cannot answer, the variant fails and you check by hand.

Retries

Retries are domain state, not a Dagster `RetryPolicy`: the attempt counter and `next_action_at` live on the variant, so every retry respects the deadline and the reconciliation rule. Retryable errors back off exponentially with jitter (30 s, 2 min, 8 min), `RateLimited` waits for the platform's `Retry-After`, and nothing retries past `max_lateness`.

Concurrency

`dagster.yaml` caps the run queue with tag limits: one run per platform account at a time, one render at a time because ffmpeg is CPU-heavy, and eight runs overall. A slow YouTube upload never delays an Instagram publish, because they hold different tags.

Platform adapters

Every platform sits behind the same `Publisher` interface, and the capability matrix below is the only place they differ. Two platforms hold posts natively, four are prepared early and published at the slot, and Reddit starts in assisted mode because its API access is gated.

Platform	Path	Prepare step	Media	Confirmed by	Access gate	Limits that shape the design
Facebook Page	Native	Upload with <code>published=false</code> and <code>scheduled_publish_time</code>	Upload	Reading the post back after the slot	Meta App Review, <code>pages_manage_posts</code>	Native window is 10 minutes to 30 days ahead (docs)
YouTube	Native	Resumable upload as private with <code>status.publishAt</code>	Upload	<code>videos.list</code> status after the slot	API compliance audit; OAuth app in production	Unaudited projects' uploads stay private; 100 uploads a day (docs)
Instagram	Prepare, then publish	Create container, poll until <code>FINISHED</code>	Public URL	Container status <code>PUBLISHED</code>	Meta App Review, <code>instagram_business_content_publish</code>	Containers expire after 24 h; rolling 24 h cap read live from <code>content_publishing_limit</code> (docs)
Threads	Prepare, then publish	Create container at least 30 s before the slot	Public URL	Container status	Meta App Review, <code>threads_content_publish</code>	250 posts per 24 h (docs)
LinkedIn	Prepare, then publish	Upload the image or video asset first	Upload	Listing the organization's recent posts	Community Management API approval	No publish-time field: a post goes live when the call is made (source)
X	Prepare, then publish	Upload media and keep the media IDs	Upload	Listing the account's recent posts	Prepaid pay-per-use credits	\$0.015 per post, \$0.20 if it contains a URL (pricing)
TikTok	Publish at the slot	None; query creator info, then init and upload at the slot	Upload	Publish-status endpoint	Content Posting audit	Unaudited apps can only post privately; privacy must be one of the creator's options (docs)
Reddit	Assisted	None; Telegram hand-off at the slot	—	You confirm with the post link	Explicit approval; commercial use needs written approval (policy)	Automated identical posts across subreddits are banned as spam

X's pricing turns into a validation rule: a caption containing a link raises a warning in the Sheet, because that one post costs over thirteen times more. Put the link in the profile instead.

Assisted mode

`AssistedPublisher` implements the same `Publisher` interface for any platform whose API is blocked or awaiting approval. At the slot it sends you a Telegram card with the caption ready to copy, the media as a short-lived link, and two buttons: Posted and Skip. Posted asks for the live URL and marks the variant `published` with you as the actor. The Sheet, calendar and digest therefore treat every platform the same, whatever its API status.

The same adapter covers TikTok until its audit passes and LinkedIn until its approval arrives. `channel_settings.mode` switches a platform between `off`, `dry_run`, `assisted` and `live` without a deploy.

Shared adapter plumbing

- One HTTP client wrapper with timeouts, structured and redacted logging, and parsing of rate-limit headers.
- Connection errors before a request is sent are retried inside the wrapper; a publish request that was sent is never retried there.
- Every platform error maps into one of five domain errors: `Retryable`, `RateLimited`, `AuthFailed`, `Rejected`, `UnknownOutcome`.
- API versions are pinned in config (the Meta Graph version, LinkedIn's version header), with a quarterly upgrade task.

Adding a platform

1. Apply for API access and record the scopes and limits in `platforms.yaml`.
2. Write the adapter: `validate`, `prepare`, `publish`, `find_live`, plus `NativeScheduler` if the platform supports it.
3. Declare its `Capabilities` and add rendition profiles.
4. Pass the shared contract suite with recorded fixtures, then add a sandbox account to the nightly run.
5. Add the Sheet tab and `_lists` entries, and register the adapter.
6. Start in `dry_run`, then move to `assisted` or `live`.

Credentials and secrets

Every secret has exactly one home: static app secrets live encrypted in git with SOPS and age, and per-account OAuth tokens live encrypted in Postgres. No secret ever appears in the Sheet, the image, the logs or a plaintext file in the repo.

Secret	Lives in	Readable by	Rotation
Google service account key (Sheets and Drive)	<code>secrets/prod.enc.yaml</code> , SOPS + age	<code>code-location</code> at start-up	Every 90 days
Platform app client IDs and secrets (Meta, Google OAuth, LinkedIn, X, TikTok, Reddit)	Same file	<code>code-location</code> , <code>connect</code>	On suspicion of leak
Telegram bot token	Same file	<code>code-location</code> , <code>connect</code>	On suspicion of leak
Postgres role passwords, one role per service	Same file	Each service its own	Yearly
Token encryption key	Same file	The vault adapter only	Yearly, by re-encrypting under a new <code>key_version</code>
Dead-man's switch ping URL	Same file	<code>code-location</code>	Rarely
OAuth access and refresh tokens, per account	<code>credentials</code> table, AES-256-GCM	The vault adapter only	Refreshed automatically before expiry
age private key	<code>/etc/dk/age.key</code> on the VPS (root only) and your password manager	The deploy script	With the VPS

At deploy, the script on the VPS decrypts the SOPS file into Docker Compose secrets, which containers read from `/run/secrets` at start-up. CI never decrypts production secrets, and secrets are never passed as build arguments.

Token encryption

Tokens are encrypted with AES-256-GCM using a fresh random nonce each time. The `account_id` is bound in as associated data, so a ciphertext copied onto another row will not decrypt. Plaintext tokens exist only in memory inside an adapter call, and a logging filter redacts anything shaped like a token.

Google access

Sheets and Drive use a service account, so there is no Google login to expire. Create one Google Cloud project, enable the Sheets and Drive APIs, and create the service account. Share the spreadsheet with its email as Editor and the media folder as Viewer, and grant only the `spreadsheets` and `drive.readonly` scopes.

YouTube is different: it needs user consent for the channel through an OAuth client. Move that consent screen to "In production" before go-live, because in Testing mode Google's refresh tokens expire after 7 days ([Google](#)).

Connecting an account

1. Run `dk connect instagram`; it prints a signed link valid for 10 minutes.
2. Open it, and the `connect` service sends you to the platform's consent screen with a `state` value and PKCE.
3. The platform redirects back to `/oauth/<platform>/callback`; the service exchanges the code, encrypts the tokens and upserts the account.
4. Telegram confirms the account, the granted scopes and the expiry date.

For the future product, the same flow becomes each tenant's Connect accounts page.

Keeping tokens alive

The hourly `refresh_tokens` job renews any token in its last fifth of life, using each adapter's declared lifetimes. A refresh failure marks the account `needs_reauth`, pauses its variants and sends you a fresh connect link. Platforms without refresh tokens for your app, such as LinkedIn's 60-day tokens, appear in the daily digest 14 days before they lapse.

Approvals and AI agents

Ticking Ready approves your own posts; agent-written posts wait for a tap in Telegram. Both routes end in the same `approved` state with a frozen snapshot, so the scheduler never knows or cares who wrote a post.

Author	Enters through	Approval	Visible in
You	The Sheet	Ticking <code>ready</code>	Your platform tabs and the Calendar tab
AI agent	Postgres directly, with <code>posts.source = agent</code>	Telegram card with the rendered preview: Approve, Edit or Reject	The read-only Calendar tab and Telegram

Agents never write to the Sheet. They coordinate only through Postgres, which keeps the Sheet a one-way input for people. Agent drafts pass the same validators as yours, plus a deterministic claims check: any halal or allergen statement must match the menu database exactly.

Agents add value in three places, all before approval: drafting per-platform variants from one master idea, an optional cheap review pass for tone and fit, and a weekly proposal of next week's mix. Nothing between approval and publish calls a model, and an import-linter contract forbids the publishing package from importing the LLM gateway at all.

Earned autonomy

Whether an agent post needs approval is looked up per tenant, action, platform and language. Every scope starts at level 0, where each agent post needs your tap. A scope earns auto-approval after a configurable streak of approvals without edits or incidents (start at 30), and drops back to level 0 on the first edit or incident. Reddit stays at level 0 permanently because of its anti-spam rules.

Failure modes

Every failure has an automatic first response and a named human action, and none of them can produce a duplicate post. Rows are ordered by how often each is expected.

Failure	Detected by	Automatic response	Your action
Platform timeout or 5xx before any response	Adapter maps it to <code>Retryable</code>	Back off and retry within the deadline	None, unless it expires
Rate limited (429)	<code>RateLimited</code> with <code>Retry-After</code>	Waits, then retries	None
Crash or timeout after the request was sent	<code>UnknownOutcome</code> , or a stale <code>publishing</code> row	Reconcile with the platform before anything else	Only if the platform cannot say
Token expired or revoked	<code>AuthFailed</code>	Account becomes <code>needs_reauth</code> ; its variants pause	Open the connect link from Telegram
Platform rejects the content	<code>Rejected</code> with the platform's message	Variant fails; the message goes into the Sheet	Fix the row, which creates a new draft
Media missing or renamed in Drive	Validation at sync	Variant becomes <code>invalid</code>	Upload or rename the file
Media changed after approval	Checksum differs at sync	Variant returns to <code>draft</code>	Re-approve (automatic for your own rows)
Prepared handle expiring before the slot	Housekeeping	Prepares again	None
Daily cap or quota reached	Pre-check before prepare	Holds within the deadline, then expires with an alert	Move the slot
VPS or Dagster down	Dead-man's switch	On restart, late variants publish if still within <code>max_lateness</code> , otherwise expire	Read the digest

Failure	Detected by	Automatic response	Your action
Restore from backup	The restore runbook enables safe mode	Every non-final variant with a past slot becomes <code>unknown</code> and is reconciled before anything publishes	Confirm the reconcile report
Google Sheets outage	Sync errors	Sync pauses; approved variants keep publishing from Postgres	None
Disk filling up	Disk guard	Alert at 70%, fetch pause at 85%	Purge early or resize
Platform API change	Nightly sandbox run fails	Alert; the platform can switch to <code>assisted</code>	Update the adapter

The restore row matters most. Without safe mode, last night's backup would have forgotten today's posts and published them a second time; with it, a restore becomes a reconciliation.

Observability and alerting

Postgres already records every transition and attempt, so metrics are dbt models and alerts go to Telegram. The one external piece is a dead-man's switch, because a dead VPS cannot send its own alert.

Logs

Services log structured JSON to stdout, and every line carries `tenant_id`, `variant_id`, `platform`, `attempt_id` and the Dagster run ID. A redaction filter strips tokens and secrets before anything is written. Docker log rotation caps disk use, and Dagster keeps per-run logs in its UI.

Metrics

Metric	Definition	Alert when
Publish lateness	p50, p95 and p99 of <code>published_at</code> - <code>publish_at</code> , per platform per day	p99 above 120 s two days running
Success rate	Published ÷ (published + failed + expired), per platform over 7 days	Below 98%
Uncertain outcomes	Variants entering <code>unknown</code> per week	Any
Token horizon	Days until the earliest token expiry, per account	Under 14 days
Quota headroom	Used against cap: Instagram, Threads, YouTube uploads, X credit balance	Above 80%
Disk usage	<code>/srv/media</code> fill level	Above 70%

Alerts

Alerts go to Telegram, deduplicated per variant and kind for one hour, each with a link to its runbook. They fire for a failed, expired or uncertain variant, an account needing re-auth, media still missing 6 hours before its slot, a quota near its cap, disk pressure, and three failed syncs in a row.

Dead-man's switch

The housekeeping schedule pings an external heartbeat URL every 5 minutes. If two pings are missed, the external monitor alerts you by email and push. A hosted monitor such as Healthchecks.io works; it is open source too, if you ever want to run it on a second machine.

Daily digest

At 08:00 Berlin time you get today's slots per platform, yesterday's results with their lateness, and everything waiting on you: approvals, re-auth links, failures, tokens expiring within 14 days and the X credit balance.

Security and compliance

The attack surface is one HTTPS endpoint, one WireGuard port and a database that never leaves the private network. Everything else follows least privilege.

Area	Control
Host	SSH keys only, root login off, SSH reachable only over WireGuard; firewall allows 80, 443 and 51820/udp; automatic security updates; fail2ban
Containers	Non-root users, read-only root filesystems where possible, no Docker socket mounted, images rebuilt weekly
Network	Postgres and the Dagster UI on the internal network and WireGuard only; TLS everywhere through Caddy
Platform scopes	Publishing scopes only; no DM, ads or follower-data scopes until a feature needs them
Media exposure	Random-token URLs, no directory listing, links live 2 hours at most, metadata stripped
Audit	Append-only events and attempts record who approved what, what was sent and when
Supply chain	Locked dependencies with uv, Renovate for updates, a Trivy image scan in CI that fails on critical vulnerabilities
Backups	Encrypted with restic and stored off the VPS in another EU location

GDPR

The pipeline handles business content and the business's own account tokens. It stores no follower or customer personal data, so the Article 9 schema rule holds by construction. Use an EU region for the VPS and backups, keep the VPS provider's and Google's data processing agreements on file, and list the system in your record of processing activities.

Platform terms

Each platform's terms are part of the design, not an afterthought. TikTok's posting flow carries a commercial content disclosure, which posts promoting your own business should set, and Reddit bans automated identical posts across communities. Keep each app review approval, audit result and policy version in the repo's `docs/compliance/` folder so the next review starts from evidence.

Testing strategy

Five test levels run in CI and nightly. The shared contract suite keeps eight adapters substitutable, and the nightly sandbox run catches platform API changes before a real post does.

Level	Covers	Tools	Runs
Unit	State machine, planning, validation rules, time zone and daylight-saving edges	pytest, Hypothesis	Every commit
Use case	Each use case against in-memory fakes of every port	pytest	Every commit
Contract	One suite every adapter must pass, including error mapping (429 to <code>RateLimited</code> and so on)	pytest with recorded HTTP fixtures	Every commit
Integration	Repositories, migrations up and down, compare-and-set under two racing runs	Postgres in testcontainers	Every commit
Sandbox	Real posts to test accounts, deleted afterwards	Private Instagram account, unlisted YouTube channel, protected X account, LinkedIn test page, private TikTok	Nightly and before each release

Property-based tests state the invariants directly: no sequence of events reaches `published` without `approved`, and `next_action_at` never falls after the deadline. A Berlin time converted to UTC and back must round-trip, except for the daylight-saving

hours, which must be rejected.

One chaos test protects the duplicate guarantee. A fault hook in the HTTP wrapper kills the run right after a publish request is sent; the test asserts the variant ends in `unknown`, reconciliation runs, and exactly one post exists.

Dry-run mode sends a platform's calls to a recording fake while everything else stays real. Use it to rehearse a full week of Sheet rows before go-live, and for every new adapter. Coverage gates require at least 90% in the domain and application layers; adapters are judged by the contract suite instead of line counts.

Deployment, CI/CD and operations

CI turns every commit into a scanned image, and one command deploys a tagged release from your laptop over WireGuard. There is no staging server: dry-run mode on production plays that role.

Pipeline

1. Every push runs `ruff`, `mypy --strict` and `import-linter`, then the unit, use-case, contract and integration tests.
2. CI builds the image with `uv`, scans it with `Trivy`, and pushes it to the GitHub container registry tagged with the commit SHA.
3. `./ops/deploy.sh v1.4.0` connects to the VPS over WireGuard and pulls that release.
4. The script waits if any publish is due within 5 minutes, runs `migrate`, then restarts the services on the new image.
5. A smoke check confirms `/healthz`, a loaded code location and ticking sensors; on failure it rolls back to the previous tag, which expand-then-contract migrations keep safe.

Keeping deploys manual is deliberate: the VPS accepts no inbound deploy hooks, and you decide when a release meets live posts.

Backups

A nightly `pg_dump` of the `app` database at 02:30 goes to EU object storage through `restic`, keeping 30 daily and 12 monthly copies. A weekly job restores the latest dump into a scratch container and checks row counts, so backups are proven, not assumed. Media is not backed up because Drive holds the originals.

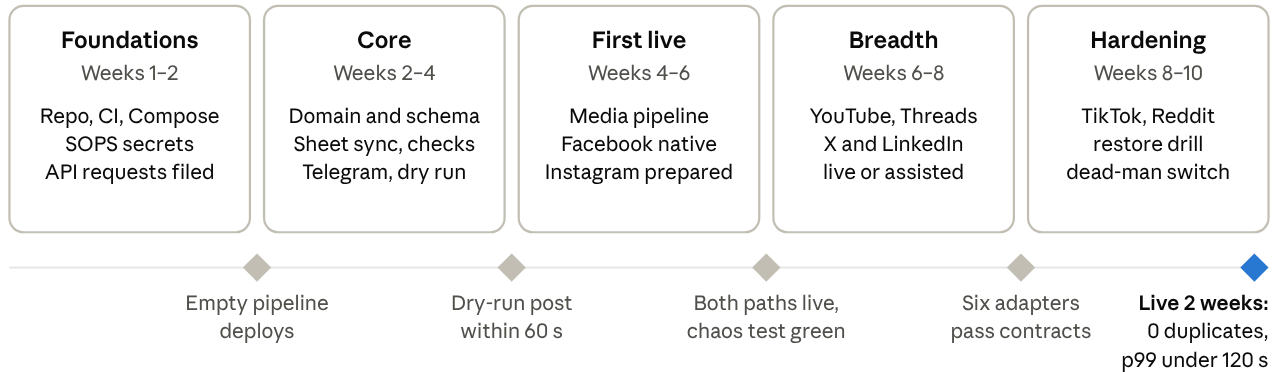
Runbooks

Task	Steps
Re-connect an account	<code>dk connect <platform></code> , open the link, approve; paused variants resume on the next tick
Pause a platform immediately	Send <code>/pause instagram</code> to the Telegram bot; <code>/resume instagram</code> undoes it
Requeue a failed variant	Edit its row in the Sheet, or run <code>dk requeue <variant_id></code>
Restore after losing the VPS	Provision from the repo, restore the dump, <code>dk safe-mode on</code> , review the reconcile report, <code>dk safe-mode off</code>
Rotate the token key	Add the new key version to SOPS, deploy, run <code>dk rekey</code> , then remove the old version
Bump a platform API version	Change the pinned version, run the contract and sandbox suites, deploy
Add a platform	Follow the checklist under Platform adapters

Delivery plan

Five phases over about ten weeks take the system from an empty repo to two weeks of clean live operation. Starting in October leaves roughly seven weeks of live running before Ramadan 2027 begins in early February (the exact date depends on the moon sighting).

Ten weeks from an empty repo to two clean weeks of live publishing



API reviews run in parallel from week 1; platforms still waiting go live in assisted mode

delivery roadmap · five phases, five gates

Each diamond is a gate: the next phase starts only when its criterion holds.

The critical path is API access, not code. Submit every application in week 1: Meta App Review with business verification, the YouTube compliance audit, LinkedIn's Community Management API, TikTok's Content Posting audit and Reddit's access request. Adapters are built against test accounts while the reviews run, and any platform still waiting at go-live runs in assisted mode, so no approval delay blocks the launch.

Architecture decision records

Twelve decisions define this design; each gets its own file under `docs/adr/` so a later change argues against the original reasoning, not against memory.

#	Decision	Rejected alternatives	Why
1	Dagster is the only scheduler	n8n, cron, Celery beat	One clock, one run history, already in the stack
2	Postgres is both record and work queue	Redis, RabbitMQ, Kafka	Volume is tiny; transactions and audit live in one place
3	The Sheet is a one-way input with status written back	Two-way sync, Sheet as database	No conflict resolution needed; the Sheet can break without harm

#	Decision	Rejected alternatives	Why
4	Ports and adapters, with Dagster as a thin shell	Logic inside Dagster ops and assets	Testable without infrastructure; Dagster stays replaceable
5	Two-phase publish, with native scheduling chosen by capability	Bespoke flow per platform	One scheduler path; adapters stay substitutable
6	At-most-once publishing with reconciliation	Blind retries	Platforms are not idempotent
7	Retries are domain state	Dagster <code>RetryPolicy</code>	Retries must respect deadlines and reconciliation
8	Google Drive carries media from the PC	rclone or Syncthing push, object storage upload	PC out of the publish path; no inbound ports at home
9	VPS disk plus tokenised Caddy URLs for media	S3-compatible storage	One vendor fewer for tenant one; the <code>MediaStore</code> port allows switching
10	SOPS and age for app secrets; AES-GCM in Postgres for tokens	Vault, Infisical, Doppler	No extra service to run and secure
11	Assisted mode as a full <code>Publisher</code>	Skipping gated platforms	The Sheet, calendar and digest stay uniform while approvals are pending
12	Default run launcher in one code-location container	Container per run	Less overhead and no Docker socket; revisit if a run needs isolation

Risks and open questions

The largest risk is a platform refusing or delaying access, and assisted mode turns that from a blocker into extra manual work. Everything else is covered by a switch, a guard or a runbook.

Risk	Impact	Mitigation
TikTok audit or Meta review delayed or refused	Channel stays private or unavailable	Apply in week 1, keep evidence in docs/compliance/, run assisted mode meanwhile
TikTok does not accept a Telegram approval card as the posting interface	Audit fails	Show creator nickname, privacy choice, interaction toggles and the commercial disclosure on the card; fall back to uploading drafts you finish in the TikTok app
Reddit refuses API access	No automated Reddit posting	Assisted mode permanently; Reddit is a low-priority channel for a cloud kitchen
Platform pricing or API changes, as X's did repeatedly in 2026	Cost or feature shifts	Per-platform kill switch, X credit alert, link-free X captions, pinned API versions
Single VPS failure	Hours without publishing	Dead-man's switch, 2-hour rebuild runbook, late posts expire instead of flooding
Human slips in the Sheet	Wrong or missing posts	Strict validation, the bulk-cancel guard, Telegram previews before approval

Open questions

- ☐ Media transport: keep Google Drive, or move to a direct PC-to-VPS sync?
- ☐ The exact accounts per platform: Facebook Page, Instagram professional account, LinkedIn company page, X handle, YouTube channel, TikTok business account, any subreddit.
- ☐ Default `max_lateness` : 2 hours everywhere, or tighter for time-bound posts such as iftar?
- ☐ Is ticking Ready enough approval for your own posts, or do you also want a Telegram confirmation?
- ☐ VPS provider and EU region, and the backup storage location.

Sources

Platform facts were checked on the official pages below on 1 October 2026.

- [Instagram content publishing](#)
- [Facebook Pages API: posts](#)
- [YouTube Data API: videos.insert](#)
- [Threads API: posts](#)
- [X API pricing](#)
- [TikTok Content Posting API: direct post](#)
- [Reddit Responsible Builder Policy](#)
- [LinkedIn API and scheduling](#)
- [Google OAuth testing-mode refresh tokens](#)
- [Dagster run launchers](#)